

PROMELA

גרא וייס
המחלקה למדעי המחשב
אוניברסיטת בן-גוריון





שפות המידול

מערכת מעברים

גרף תוכנית

שזירת
מערכות
מעברים

מעגל
חומרה

מערכת
ערוצים

שזירה עם
תקשורת
סנכרונית

שזירה בלי
תקשורת

Promela



Spin Version 6.2.5 -- 3 May 2013 :: iSpin Version 1.1.0 -- 7 June 2012

Edit/View Simulate / Replay Verification Swarm Run <Help> Save Session Restore Session <Quit>

Safety	Storage Mode	Search Mode
☒ safety ☒ + invalid endstates (deadlock) ☒ + assertion violations ☐ + xr/xs assertions ☐ Liveness ☐ non-progress cycles ☐ acceptance cycles ☐ enforce weak fairness constraint	☒ exhaustive ☐ + minimized automata (slow) ☐ + collapse compression ☐ hash-compact ☐ bitstate/supertrace ☐ Never Claims ☒ do not use a never claim or ltl property ☐ use claim claim name (opt):	☒ depth-first search ☒ + partial order reduction ☐ + bounded context switching with bound: 0 ☐ + iterative search for short trail ☐ breadth-first search ☒ + partial order reduction ☒ report unreachable code

Run Stop Save Result in: pan.out

```

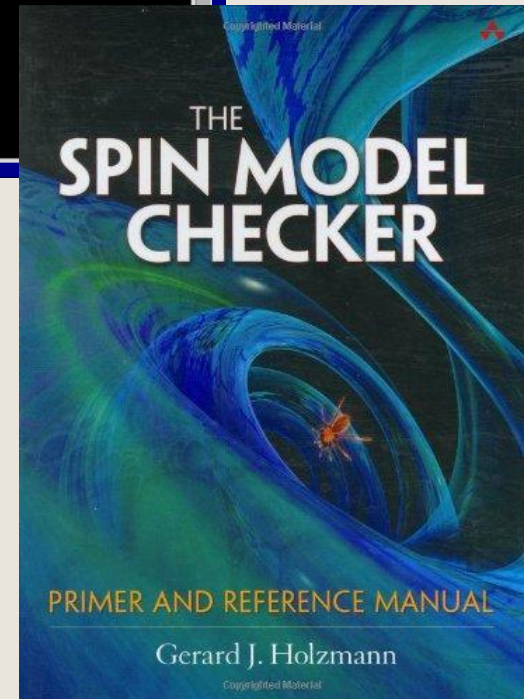
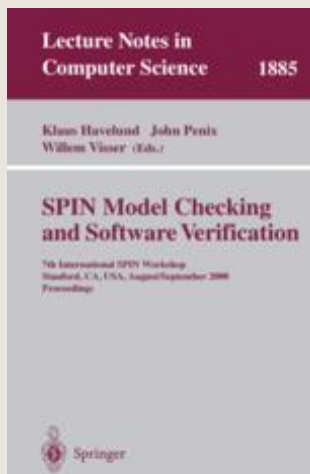
1 byte sema;
2
3 active proctype pBonsolr()
4 {
5     sema == 0;
6     printf("Bonsolr");
7     sema++;
8 }
9 active proctype pChance()
10 {
11     sema == 1;
12     printf(" et bonne chance\n");
13     sema--;
14 }
15 init
16 {
17     sema = 0;
18 }
  
```

verification result:
 spin -a test3.pml
 gcc -DMEMLIM=1024 -O2 -DXUSAFE -DSAFETY -DNOCLAIM -w -o pan pan.c
 ./pan -m10000
 Pid: 5635
 pan:1: Invalid end state (at depth 4)
 pan: wrote test3.pml.trail

(Spin Version 6.2.5 -- 3 May 2013)
 Warning: Search not completed
 + Partial Order Reduction

Full statespace search for:
 never claim - (not selected)
 assertion violations +
 cycle checks - (disabled by -DSAFETY)
 invalid end states +

State-vector 28 byte, depth reached 10, errors: 1



Promela



PROMELA (**Process** or **Protocol Meta Language**) is a **verification modeling language**

introduced by **Gerard J. Holzmann**.



The language allows for the dynamic creation of **concurrent** processes to model, for example, **distributed systems**.

In PROMELA models, communication via message channels can be defined to be synchronous (i.e., rendezvous), or asynchronous (i.e., buffered).

PROMELA models can be analyzed with the **SPIN model checker**, to verify that the modeled system produces the desired behavior.

nanoPromela

אנחנו נדבר על **גרסה מוקטנת** שנקרא לה nano PROMELA



- משתנים משותפים ותקשורת מבוססת ערוצים

- הסמנטיקה הפורמאלית מבוססת על מערכות ערוצים

- תהליכים מוגדרים באמצעות שפת פקודות מותנות

- אין הגדרה מפורשת של הפעולות. מוגדרות באמצעות הפקודות.



שפת מפרט עם סמנטיקה מוגדרת באופן מתמטי

- נתאר "שפת תכנות" למודלים
- המטרה: לאפשר למהנדסים לבנות מודל אותו ניתן יהיה לנתח באמצעות כלים פורמאליים
- נקודה מרכזית: אנחנו דורשים שלמפרט תהיה סמנטיקה פורמאלית!
- במקרה של Promela: סמנטיקה מוגדרת ע"י תרגום למערכת ערוצים ודרכה למערכת מעברים

לא ניתן להוכיח נכונות ללא סמנטיקה פורמאלית מוגדרת היטב



תוכנית ב-nano Promela

- אוסף תהליכים $P_1, \dots, P_n$ המורכבים ע"י שזירה
- כל תהליך כתוב סדרתית כרצף צעדים:

דילוג למצב הבא	<i>skip</i>
השמת ערך למשתנה	<i>x := expr</i>
הוצאת הודעה מתור ערוץ תקשורת	<i>c?x</i>
כתיבת הודעה לתור ערוץ תקשורת	<i>c!expr</i>
תנאי	<i>if-fi</i>
לולאה	<i>do-od</i>

- השפה מאפשרת אי-דטרמיניזם – אנחנו בונים מודל, לא מתכנתים



דוגמה: האלגוריתם של פטרסון

קוד עבור תהליך P_1 :

```
do :: true ->
    /* Noncritical Section */
    atomic { b1:=true; x:=2 };
    if ::(x==1) ∨ ¬b2 -> crit1:=true fi;
    /* Critical Section */
    atomic { crit1:=false; b1:=false }
od
```




דוגמה: מכונת מכירת משקאות

```
do
  :: true -> skip; /* Coin Insertion */
    if :: ns > 0          -> ns:=ns-1
      :: nc > 0          -> nc:=nc-1
      :: nc==0 && ns==0 -> skip
    fi
  :: true -> atomic {nc:=10; ns:=10}
od
```



nanoPromela

תוכנה $Pr = [P_1 | \dots | P_n]$ כאשר P_i הם תהליכים הרצים במקביל
תהליך מוגדר ע"י פקודה:

```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':' '=' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {' (VARNAME ':' '=' intexpr ';' ) * VARNAME ':' '=' intexpr '}';
```

```
option: '::' boolexpr '->' stmt ;
```



עץ הפריסה

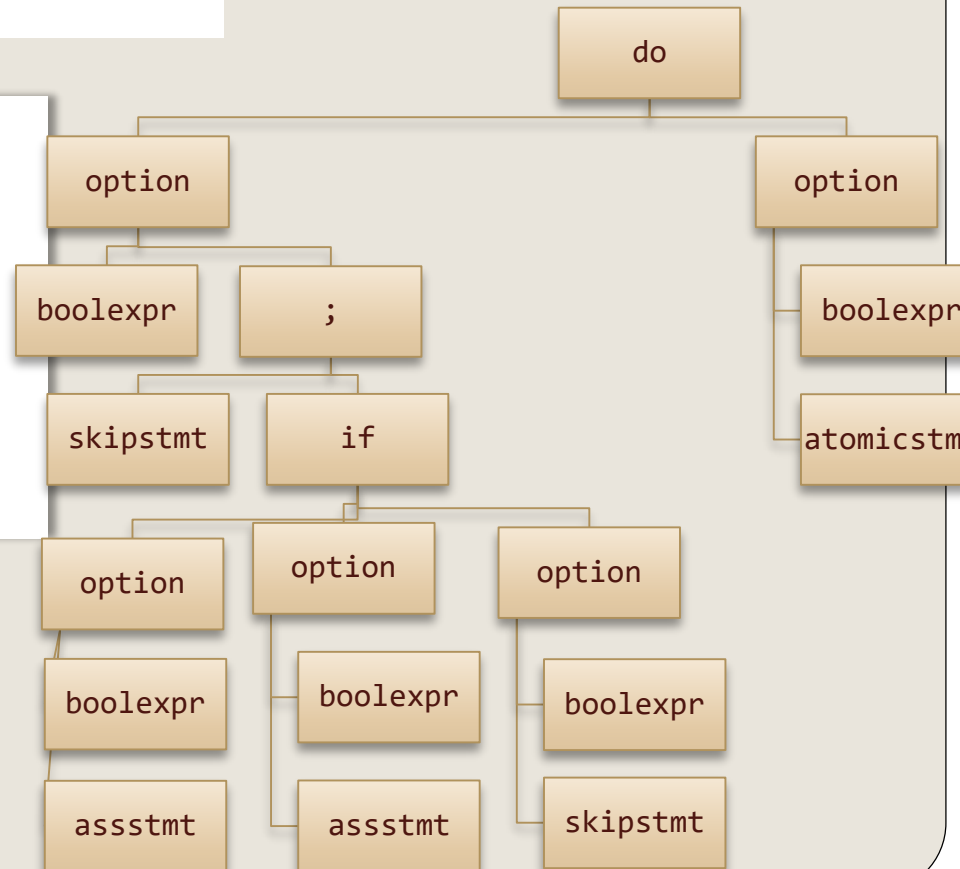
```
stmt: ifstmt | dostmt | assstmt | chanreadstmt | chanwritestmt |  
      atomicstmt | skipstmt | stmt ';' stmt ;
```

```
ifstmt      : 'if' option+ 'fi' ;  
dostmt      : 'do' option+ 'od' ;  
assstmt     : VARNAME ':= ' intexpr ;  
chanreadstmt : CHANNAME '?' VARNAME ;  
chanwritestmt : CHANNAME '!' intexpr ;  
skipstmt    : 'skip' ;
```

```
atomicstmt :  
  'atomic {' (VARNAME ':= ' intexpr ';')* VARNAME ':= ' intexpr '}';
```

```
option: '::' boolexpr '->' stmt ;
```

```
do  
  :: true -> skip; /* Coin Insertion */  
  if :: ns > 0      -> ns:=ns-1  
    :: nc > 0      -> nc:=nc-1  
    :: nc==0 && ns==0 -> skip  
  fi  
  :: true -> atomic {nc:=10; ns:=10}  
od
```





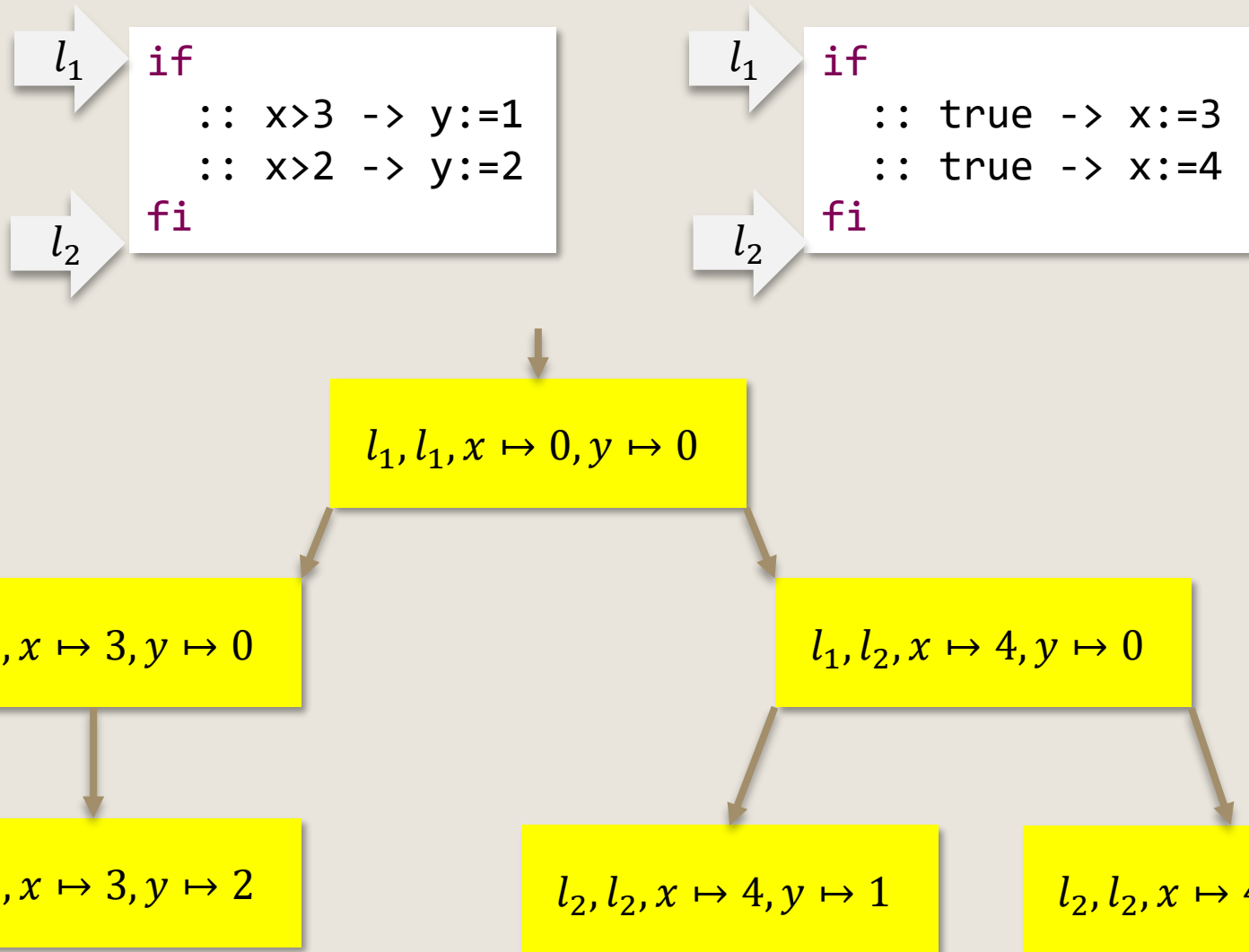
פקודות תנאי

$$\text{if } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ fi}$$

- בחירה אי דטרמיניסטית בין פקודות $stmt_i$ שהתנאים g_i שלהן מתקיימים
- סמנטיקה של **test-and-set**: בדיקת התנאי + בחירת הפקודה + ביצוע הצעד הראשון של הפקודה מתבצעים באופן אטומי.
- פקודת if-fi נתקעת אם אף אחד מהתנאים אינו מתקיים
- תהליכים מקבילים יכולים לשחרר את החסימה ע"י שינוי משתנים משותפים
- למשל: כאשר $y = 0$, הפקודה **if ::y>0 -> x := 39 fi** ממתינה ש y יהיה גדול מאפס
- **הכללה** של שפות תכנות רגילות:
 - **if ::g→stmt₁ ::¬g→stmt₂ fi** שקול ל- **if g then stmt₁ else stmt₂ fi**
 - **if ::g→stmt₁ ::¬g→skip fi** שקול ל- **if g then stmt₁ fi**



פקודות תנאי





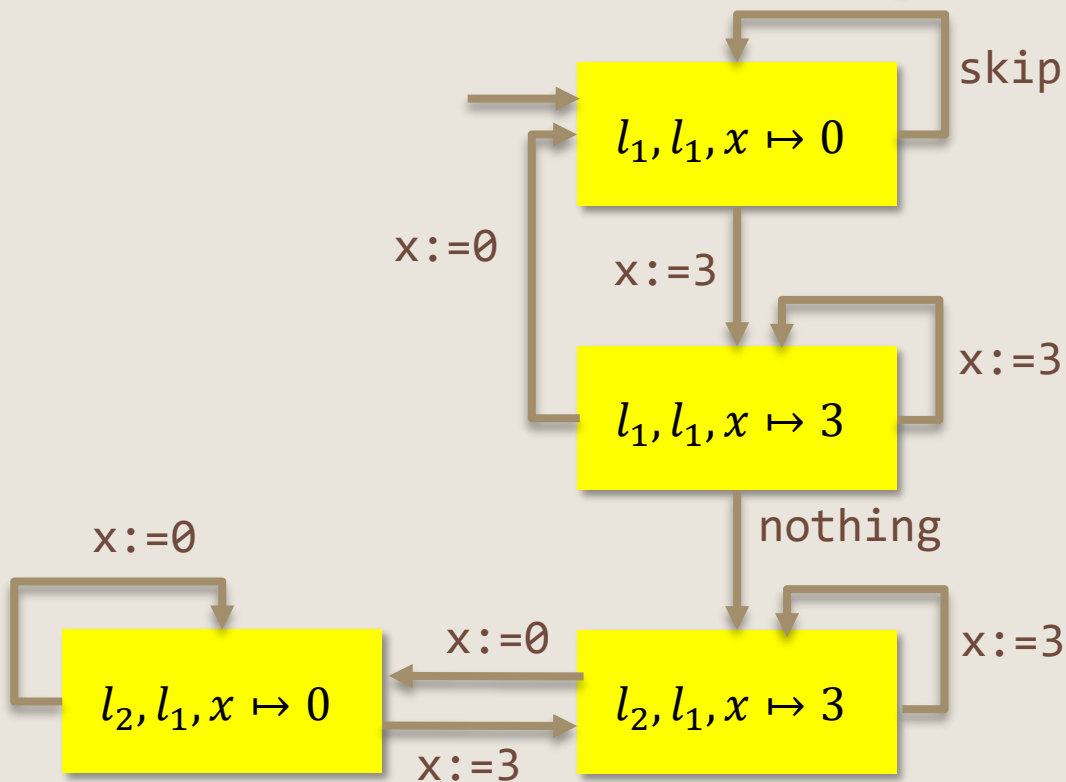
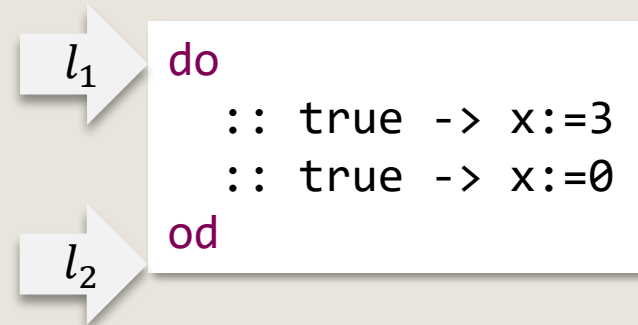
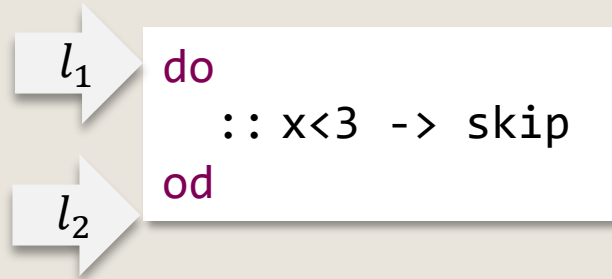
פקודות לולאה

$$\text{do} :: g_1 \rightarrow stmt_1 \cdots :: g_n \rightarrow stmt_n \text{ od}$$

- ביצוע חוזר של כל הפקודות $stmt_i$ שתנאיהן g_i מתקיימים
- אין תקיעה כשכל התנאים מופְּרים - הלולאה מסתיימת
- הכללה של שפות תכנות רגילות:
- $\text{do} :: g \rightarrow stmt \text{ od}$ שקול ל- $\text{while } g \text{ do } stmt \text{ od}$
- אין פקודת break ליציאה מלולאות



פקודות לולאה





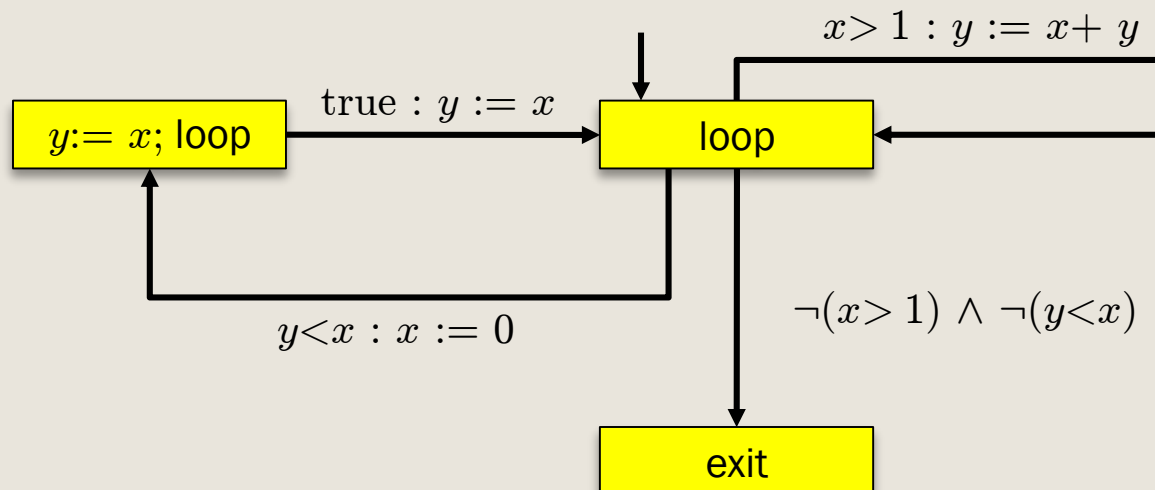
הבהרה בקשר לסמנטיקה של do

Promela

```
do  :: atomic{x > 1 -> y:=x+y}
    :: atomic{y < x -> x:=0}; y:=x
    :: atomic{else -> break}
od
```

nanoPromela

```
do  :: x > 1 -> y:=x+y
    :: y < x -> x:=0; y:=x
od
```





אימות אלגוריתם למניעה הדדית

```
boolean array b(0;1) integer k,  
process i, with i either 0 or 1;  
C0:    b(i) := false;  
C1:    if k != i then begin  
C2:    if not (b(1-i)) then go to C2;  
        else k := i; go to C1 end;  
        else critical section;  
        b(i) := true;  
        remainder of program;  
        go to C0;
```



פרוטוקול שהוצע למניעה הדדית

"Comments on a Problem in Concurrent Programming Control," by H. Hyman
Communications of the ACM, 1966, Volume 9

```
int cnt, k, b[2];  
  
active [2] proctype P() {  
    assert(_pid == 0 || _pid == 1);  
    C0: b[_pid] = 0;  
    C1: if  
        :: (k != _pid) ->  
            (b[1 - _pid]);  
            k = _pid;  
            goto C1  
  
        :: else ->  
            cnt = cnt+1;  
            assert(cnt == 1); /* crit. sect. */  
            cnt = cnt-1;  
            b[_pid] = 1;  
  
    fi;  
    goto C0  
}
```



תרגום הפרוטוקול ל-PROMELA



```

1: proc 1 (P) line 5 "hy66" (state 1) [assert((( _pid==0) || (_pid==1)))]
2: proc 1 (P) line 7 "hy66" (state 2) [b[_pid] = 0]
3: proc 1 (P) line 9 "hy66" (state 3) [(k!= _pid)]
4: proc 0 (P) line 5 "hy66" (state 1) [assert((( _pid==0) || (_pid==1)))]
5: proc 0 (P) line 7
14: proc 1 (P) line 11 "hy66" (state 5) [k = _pid]
6: proc 0 (P) line 14
15: proc 1 (P) line 14 "hy66" (state 7) [else]
7: proc 0 (P) line 15
16: proc 1 (P) line 15 "hy66" (state 8) [cnt = (cnt+1)]
8: proc 0 (P) line 16
17: proc 1 (P) line 16 "hy66" (state 9) [assert((cnt==1))]
9: proc 0 (P) line 17
18: proc 0 (P) line 15 "hy66" (state 8) [cnt = (cnt+1)]
10: proc 0 (P) line 19 spin: line 16 "hy66", Error: assertion violated
11: proc 1 (P) line 10
19: proc 0 (P) line 16 "hy66" (state 9) [assert((cnt==1))]
12: proc 0 (P) line 7 spin: trail ends after 19 steps
13: proc 0 (P) line 14 #processes: 2
                        cnt = 2
                        k = 1
                        b[0] = 0
                        b[1] = 0
19: proc 1 (P) line 17 "hy66" (state 10)
19: proc 0 (P) line 17 "hy66" (state 10)
2 processes created

```



חידה: גשר צר מאוד



• ארבעה חיילים צריכים לחצות גשר

• מהירויות החיילים: 5,10,20,25 דקות בהתאמה

• **לכל היותר**, שניים יכולים להיות על הגשר בו זמנית



• יש לפיד אחד

• מדובר בשעת לילה, הגשר צר מאוד, **חייבים ללכת עם לפיד**

• האם אפשר להעביר את כולם בתוך 60 דקות?



```

byte now=0; byte time[4]; bit where[4]; bit torch; byte i,j;
active proctype bridge() {
  time[0] = 5; time[1] = 10; time[2] = 20; time[3] = 25;
  do :: now <= 60 ->
    assert(!(where[0]==1 && where[1]==1 && where[2]==1 && where[3]==1));
    if :: where[0] == torch -> i = 0
      :: where[1] == torch -> i = 1
      :: where[2] == torch -> i = 2
      :: where[3] == torch -> i = 3
    fi;
    if :: where[0] == torch -> j = 0
      :: where[1] == torch -> j = 1
      :: where[2] == torch -> j = 2
      :: where[3] == torch -> j = 3
    fi;
    where[i] = 1 - torch;
    where[j] = 1 - torch;
    torch = 1 - torch;
    now = now + MAX(time[i], time[j])
  od
}

```

Spin יחפש ריצה שתפר את ה-assertion

בחירה אי-דטרמיניסטית לקביעת
חייל שייקח את הלפיד

בחירה אי-דטרמיניסטית לקביעת
חייל נוסף שיבוא איתו

דוגמה: פרוטוקול תקשורת



```
# model of a process
```

```
in!id; out?max
```

```
# moderator
```

```
atomic {c:=0; max:=0};
```

```
do :: c < n =>
```

```
  in ? id;
```

```
  c := c + 1
```

```
  if :: id > max => max:=id
```

```
    :: id <= max => skip
```

```
fi
```

```
od;
```

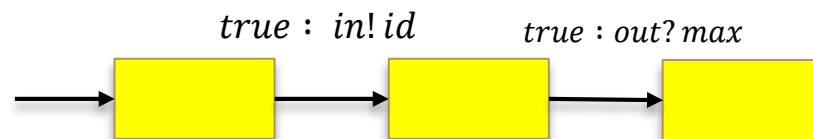
```
c := 0;
```

```
do
```

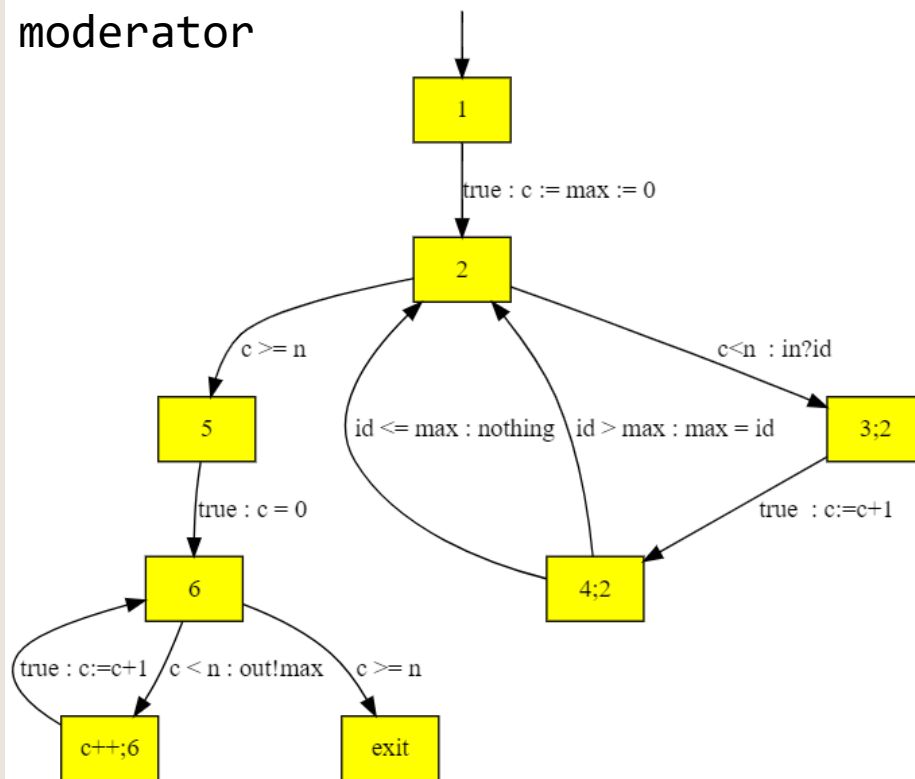
```
  :: c < n => out!max; c:=c+1
```

```
od
```

process(id)



moderator



פרוטוקול: כל תהליך שולח את המזהה שלו למתאם. בסיבוב שני, המתאם שולח בהודעות את המזהה הגבוה ביותר שהתקבל.

נעבור להגדרות פורמאליות

נגדיר את קבוצת המקומות בתוכנית באופן רקורסיבי

לאחר מכן, באמצעות כללי גזירה, נגדיר את
המעברים בין המקומות השונים



הגדרה רקורסיבית של המקומות

+

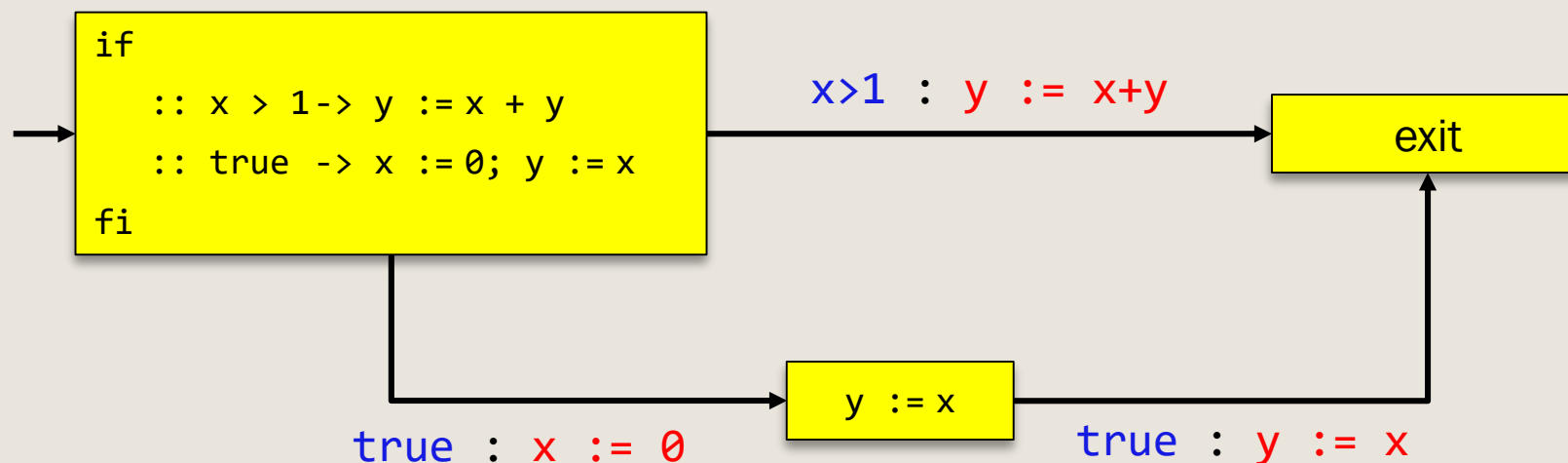


כללי גזירה להגדרת המעברים בין המקומות



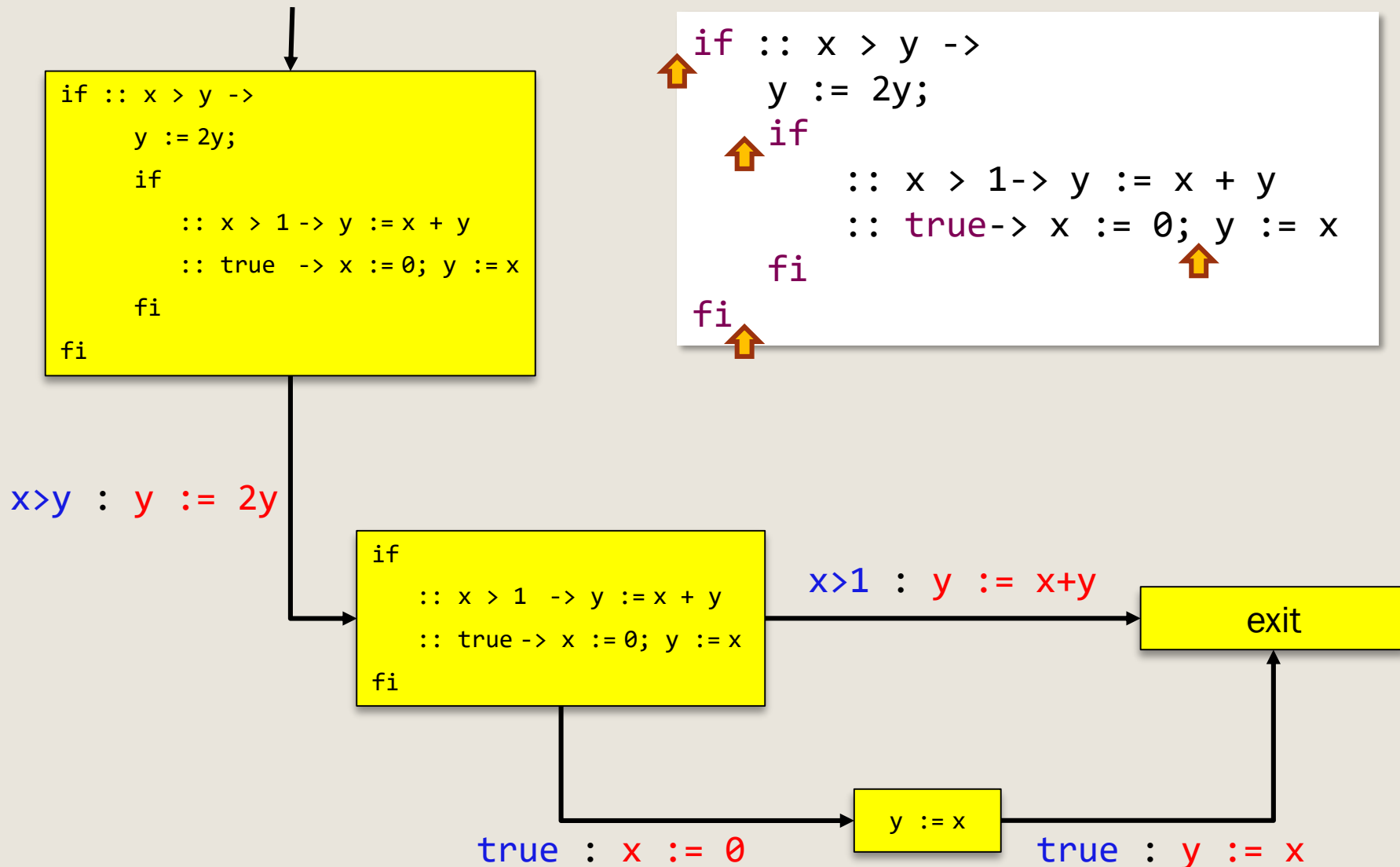
דוגמה: גרף תוכנית למבנה תנאי

```
if
  :: x > 1 -> y := x + y
  :: true  -> x := 0; y := x
fi
```





דוגמה: גרף תוכנית למבנה תנאי מקוונ



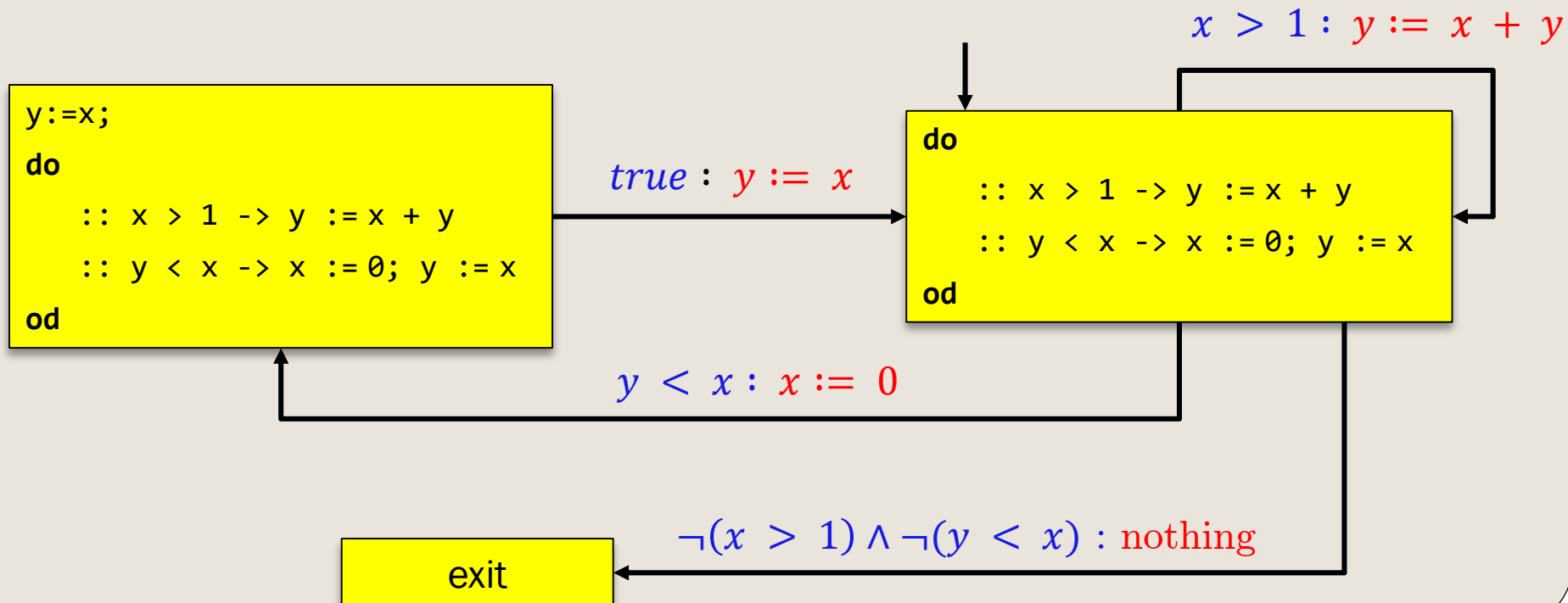


דוגמה: גרף תוכנית למבנה לולאה

```

do
  :: x > 1 -> y := x + y
  :: y < x -> x := 0; y := x
od

```





מקומות בגרף התוכנית = תת הפקודות

מגדירים את המקומות בגרף התוכנית כִּתְת הפקודות בתוכנית:

$$Loc = \text{sub}(\text{program})$$

מקרי הבסיס של ההגדרה הִרְקוּרְסִיבִית:

אם

$$\text{stmt} \in \{\text{skip}, x := \text{expr}, c?x, c!\text{expr}, \text{atomic}\{\text{assgn}\}\}$$

מגדירים את תת הפקודות להיות:

$$\text{sub}(\text{stmt}) := \{\text{stmt}, \text{exit}\}$$

המשך ההגדרה, באופן רִקוּרְסִיבִי, בשקפים הבאים...



תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות

$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \\ \{ \text{stmt}'; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

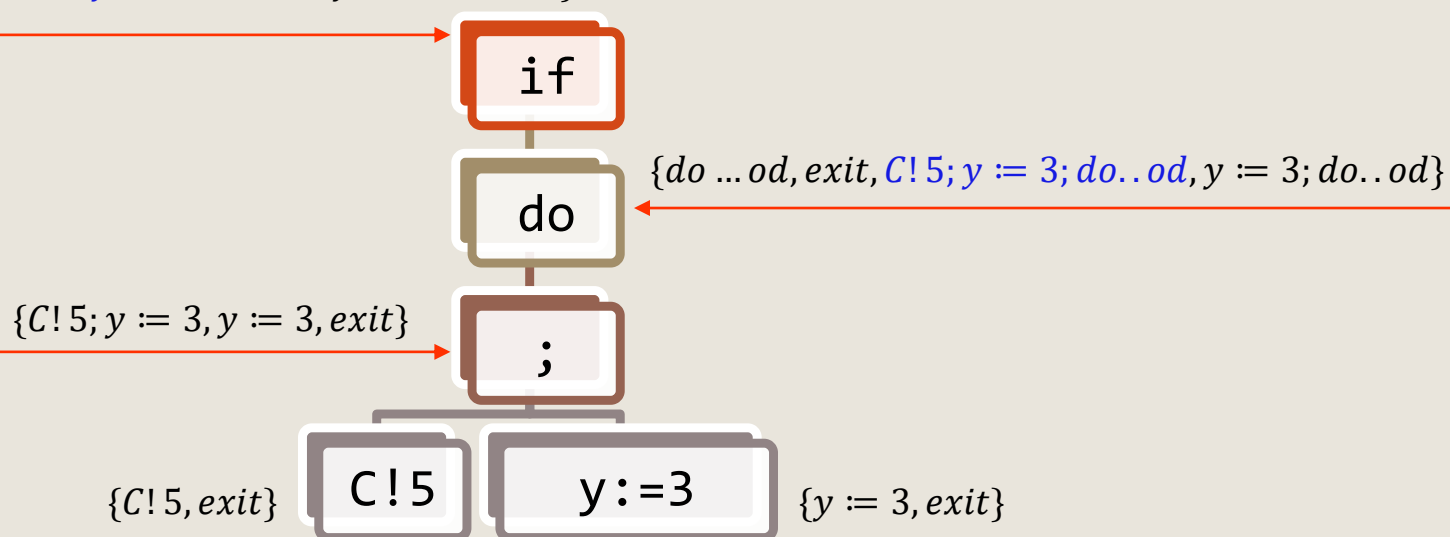
$$\text{sub} \left(\overbrace{\text{if} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ fi}}^{\text{cond_cmd}} \right) := \\ \{ \text{cond_cmd}, \text{exit} \} \cup \text{sub}(\text{stmt}_1) \cup \cdots \cup \text{sub}(\text{stmt}_n)$$

$$\text{sub} \left(\overbrace{\text{do} :: g_1 \rightarrow \text{stmt}_1 \cdots :: g_n \rightarrow \text{stmt}_n \text{ od}}^{\text{loop_cmd}} \right) := \\ \{ \text{loop_cmd}, \text{exit} \} \cup \bigcup_{i=1, \dots, n} \{ \text{stmt}' ; \text{loop_cmd} : \text{stmt}' \in \text{sub}(\text{stmt}_i) \setminus \{\text{exit}\} \}$$



$$sub\left(\overbrace{\mathbf{if} \ :: g_1 \rightarrow stmt_1 \ \cdots \ :: g_n \rightarrow stmt_n \ \mathbf{fi}}^{\text{cond_cmd}}\right) := \{\text{cond_cmd}, \text{exit}\} \cup sub(stmt_1) \cup \cdots \cup sub(stmt_n)$$

$$\{if \dots fi, exit, do \dots od, C!5; y := 3; do \dots od, y := 3; do \dots od\}$$



```
if :: x<2 -> do ::z>3 -> C!5; y:=3 od fi
```

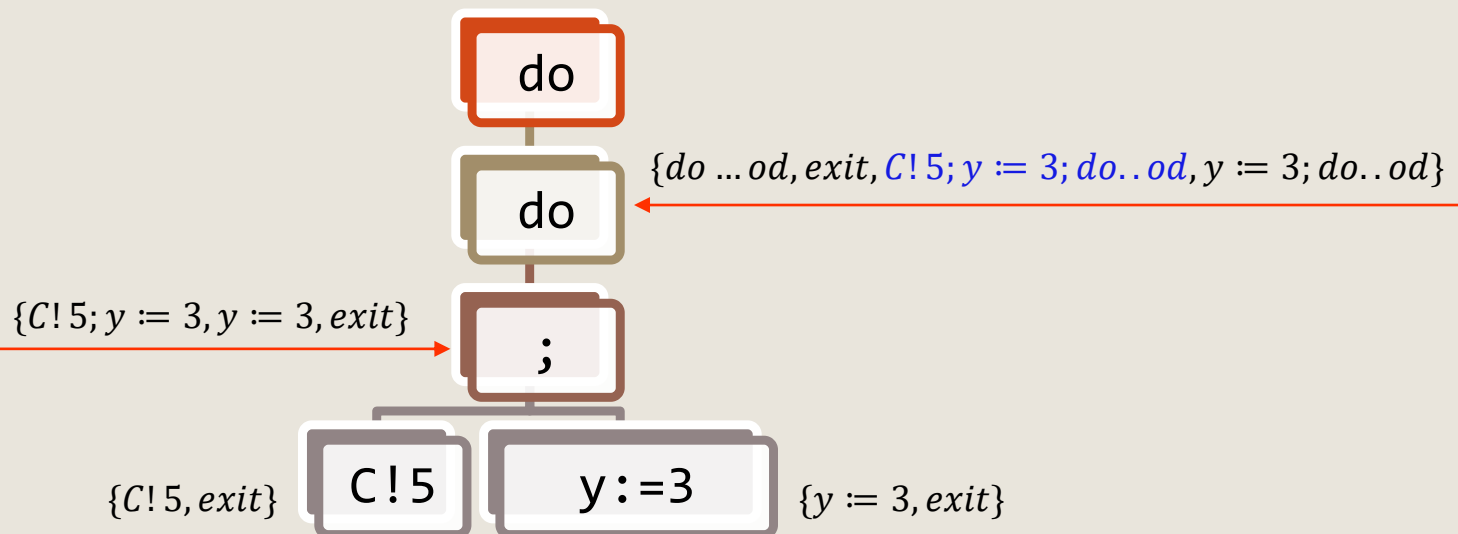
תת-פקודות של פקודות מורכבות

הגדרות רקורסיביות

$$\text{sub}(\text{stmt}_1; \text{stmt}_2) := \{ \text{stmt}' ; \text{stmt}_2 : \text{stmt}' \in \text{sub}(\text{stmt}_1) \setminus \{\text{exit}\} \} \cup \text{sub}(\text{stmt}_2)$$

$$\text{sub} \left(\overbrace{\text{if } g_1 \rightarrow \text{stmt}_1 \cdots g_n \rightarrow \text{stmt}_n \text{ fi}}^{\text{cond_cmd}} \right) := \{\text{cond_cmd}, \text{exit}\} \cup \text{sub}(\text{stmt}_1) \cup \cdots \cup \text{sub}(\text{stmt}_n)$$

$$\text{sub} \left(\overbrace{\text{do } g_1 \rightarrow \text{stmt}_1 \cdots g_n \rightarrow \text{stmt}_n \text{ od}}^{\text{loop_cmd}} \right) := \{\text{loop_cmd}, \text{exit}\} \cup \bigcup_{i=1, \dots, n} \{ \text{stmt}' ; \text{loop_cmd} : \text{stmt}' \in \text{sub}(\text{stmt}_i) \setminus \{\text{exit}\} \}$$



do :: x < 2 -> do :: z > 3 -> C!5; y := 3 od od

דוגמה: חישוב *sub* באופן רקורסיבי



Sub = { *y*:=2*y*; *innerif*, *innerif*, *y*:=*x*+*y*,
x:=0; *y*:=*x*, *y*:=*x*, *exit* }

y := 2*y*;
 if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi

y := 2*y*

Sub = {*y*:=2*y*, *exit*}

if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi
innerif

y := *x* + *y*

Sub = {*y*:=*x*+*y*, *exit*}

Sub = {*x*:=0, *exit*} *x* := 0

x := 0; *y* := *x*

Sub = {*x*:=0; *y*:=*x*, *y*:=*x*, *exit*}

Sub = {*y*:=*x*, *exit*}

y := *x*

if :: *x* > *y* -> *y* := 2*y*;
 if
 :: *x* > 1 -> *y* := *x* + *y*
 :: true -> *x* := 0; *y* := *x*
 fi
 fi
outerif

Sub = { *outerif*, *y*:=2*y*; *innerif*,
innerif, *y*:=*x*+*y*,
x:=0; *y*:=*x*, *y*:=*x*, *exit* }

Sub = {*innerif*, *y*:=*x*+*y*, *x*:=0; *y*:=*x*,
y:=*x*, *exit*}



מגדירים את הקשתות בגרף התוכנית

$$\frac{}{\text{skip} \xrightarrow{\text{true} : \text{nothing}} \text{exit}}$$

$$\frac{}{x := \text{expr} \xrightarrow{\text{true} : x := \text{expr}} \text{exit}}$$

$$\frac{}{C?x \xrightarrow{\text{true} : C?x} \text{exit}}$$

$$\frac{}{C!x \xrightarrow{\text{true} : C!x} \text{exit}}$$

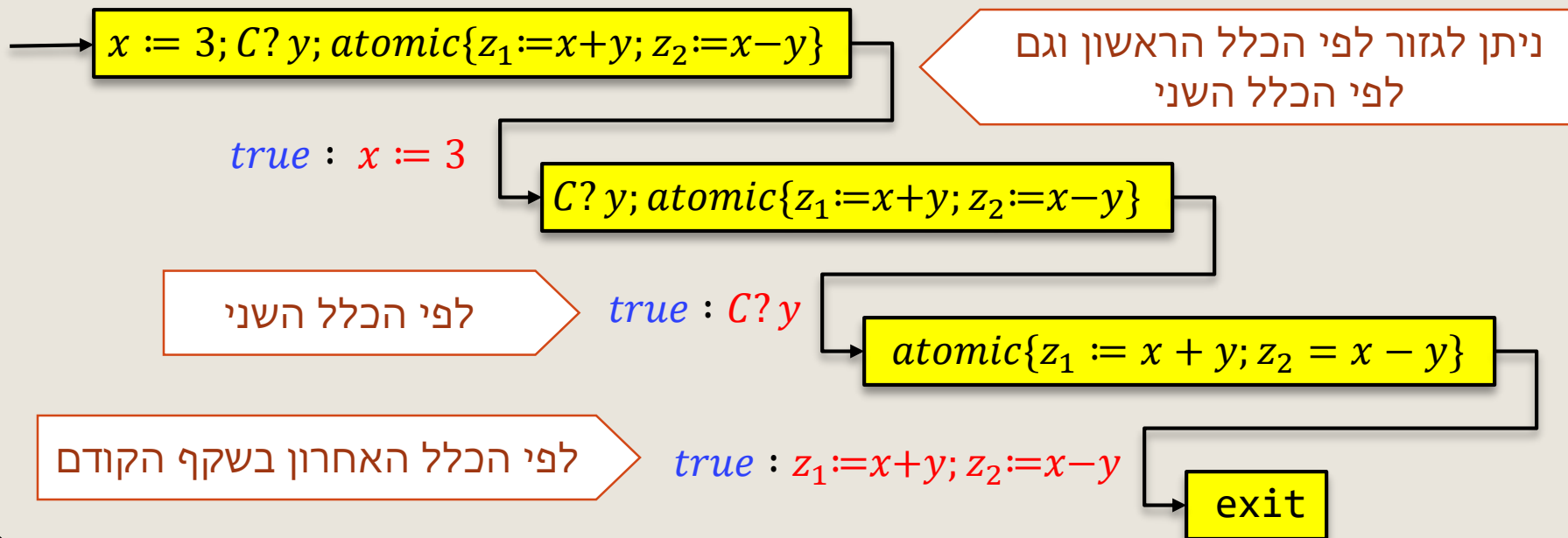
$$\frac{}{\text{atomic}\{x_1 := \text{exp}_1; \dots x_m := \text{exp}_m\} \xrightarrow{\text{true} : x_1 := \text{exp}_1; \dots x_m := \text{exp}_m} \text{exit}}$$

כללי הסק לשרשור



$$\frac{stmt_1 \xrightarrow{g:\alpha} stmt'_1 \quad \wedge \quad stmt'_1 \neq \text{exit}}{stmt_1; stmt_2 \xrightarrow{g:\alpha} stmt'_1; stmt_2}$$

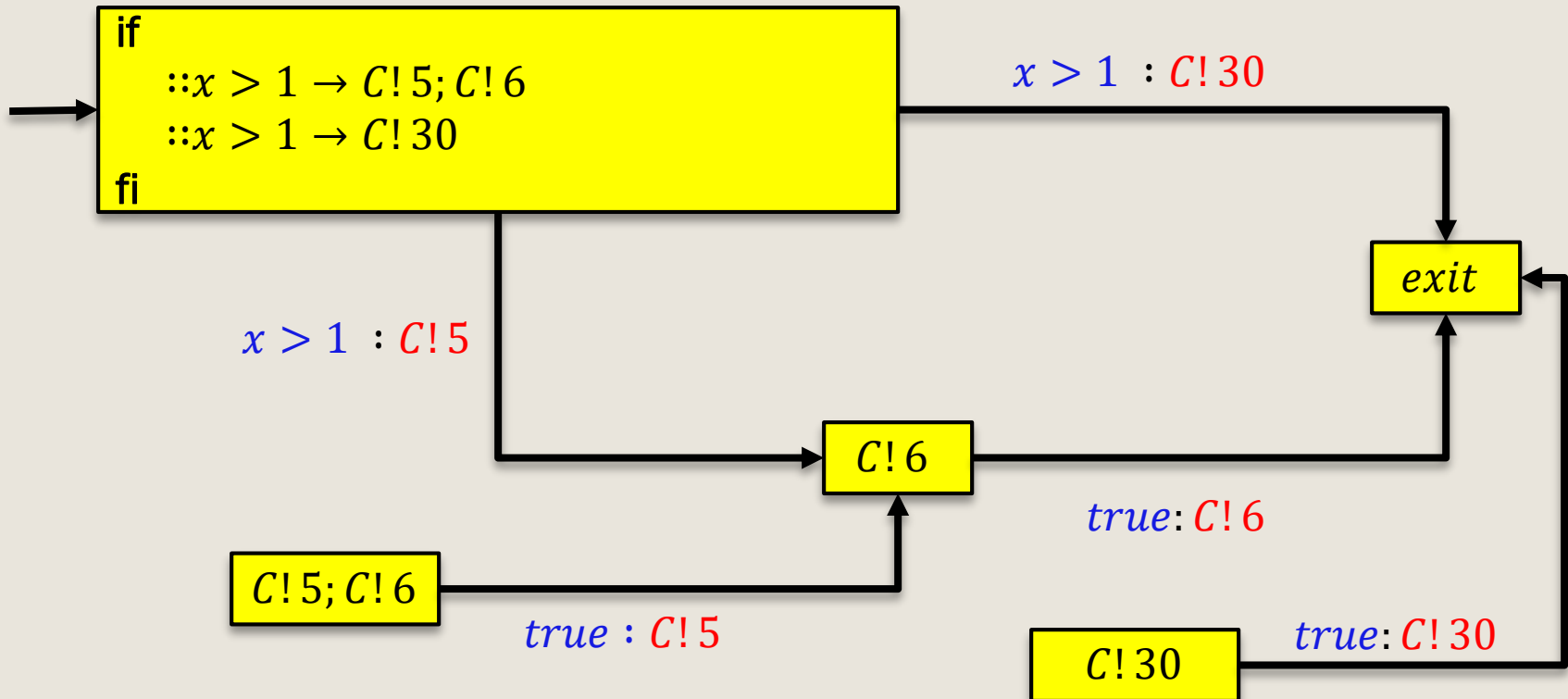
$$\frac{stmt_1 \xrightarrow{g:\alpha} \text{exit}}{stmt_1; stmt_2 \xrightarrow{g:\alpha} stmt_2}$$





כללי הסק לפקודות תנאי

$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i}{\text{if } ::g_1 \rightarrow stmt_1 \ \dots \ ::g_n \rightarrow stmt_n \text{ fi} \xrightarrow{g_i \wedge h:\alpha} stmt'_i}$$



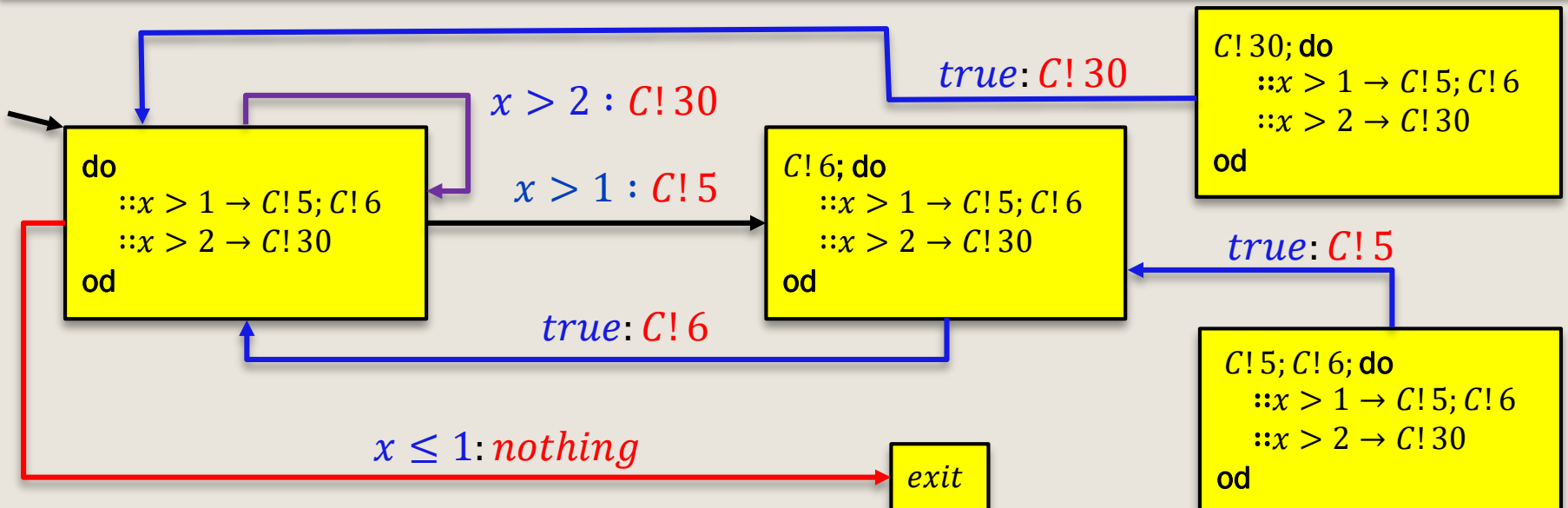
כללי הסק לפקודות לולאה



$$\frac{stmt_i \xrightarrow{h:\alpha} stmt'_i \wedge stmt'_i \neq \text{exit}}{\text{do } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ od} \xrightarrow{g_i \wedge h:\alpha} stmt'_i; \text{do } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ od}}$$

$$\frac{stmt_i \xrightarrow{h:\alpha} \text{exit}}{\text{do } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ od} \xrightarrow{g_i \wedge h:\alpha} \text{do } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ od}}$$

$$\text{do } ::g_1 \rightarrow stmt_1 \cdots ::g_n \rightarrow stmt_n \text{ od} \xrightarrow{\neg g_1 \wedge \cdots \wedge \neg g_n : \text{nothing}} \text{exit}$$





דוגמה

```
if
  ::  $x > 1 \rightarrow y := x + y$ 
  :: true  $\rightarrow x := 0; y := x$ 
fi
```



דוגמה: תת הפקודות

```
if
  ::  $x > 1 \rightarrow y := x + y$ 
  :: true  $\rightarrow x := 0; y := x$ 
fi
```

if ... fi

$y := x + y$

exit

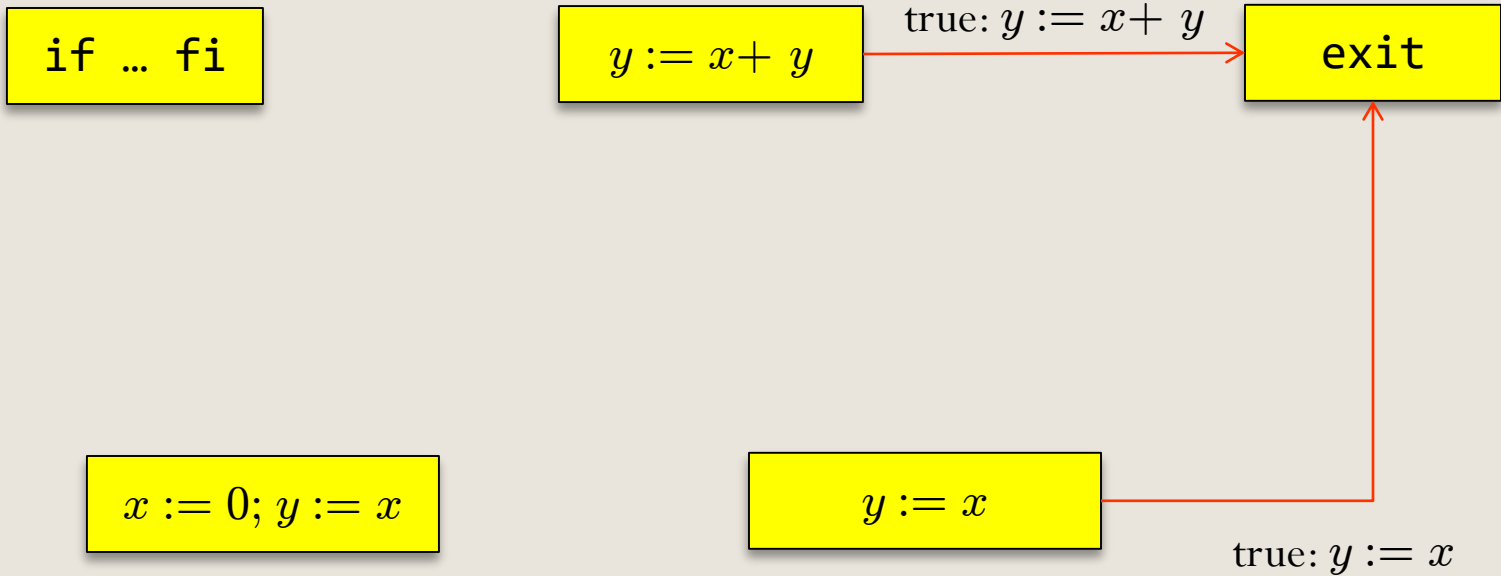
$x := 0; y := x$

$y := x$



דוגמה: מעברים לפקודות השמה

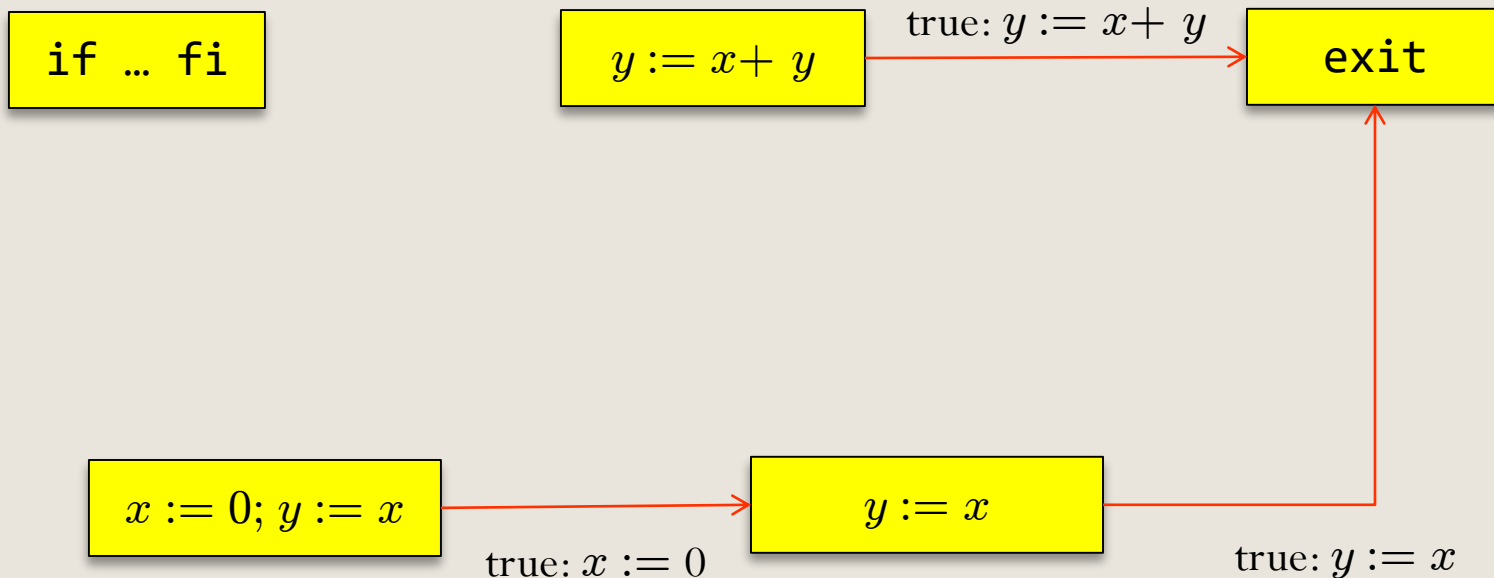
```
if
  ::  $x > 1 \rightarrow y := x + y$ 
  :: true  $\rightarrow x := 0; y := x$ 
fi
```





דוגמה: מעבר לפקודת הרכבה

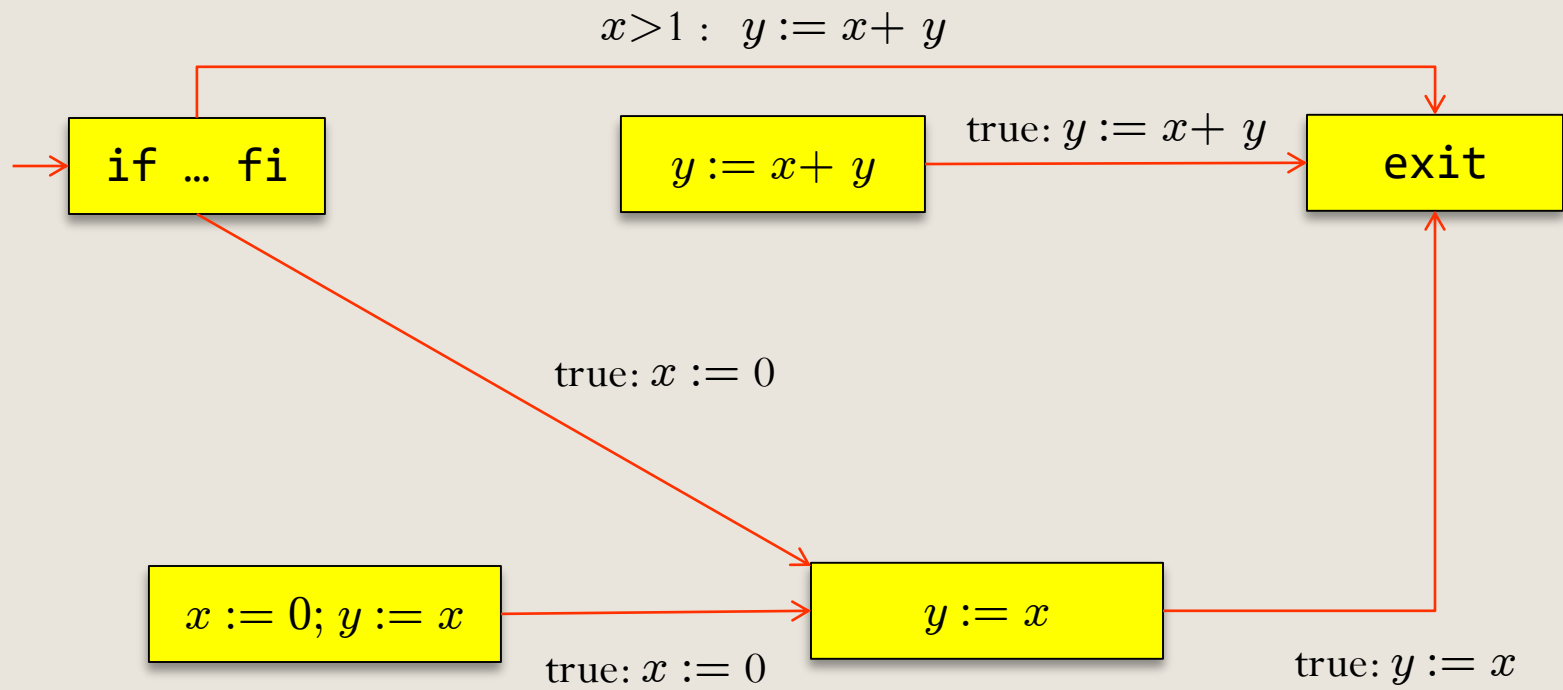
```
if  
  ::  $x > 1 \rightarrow y := x + y$   
  :: true  $\rightarrow x := 0; y := x$   
fi
```





דוגמה: מעברים לפקודת תנאי

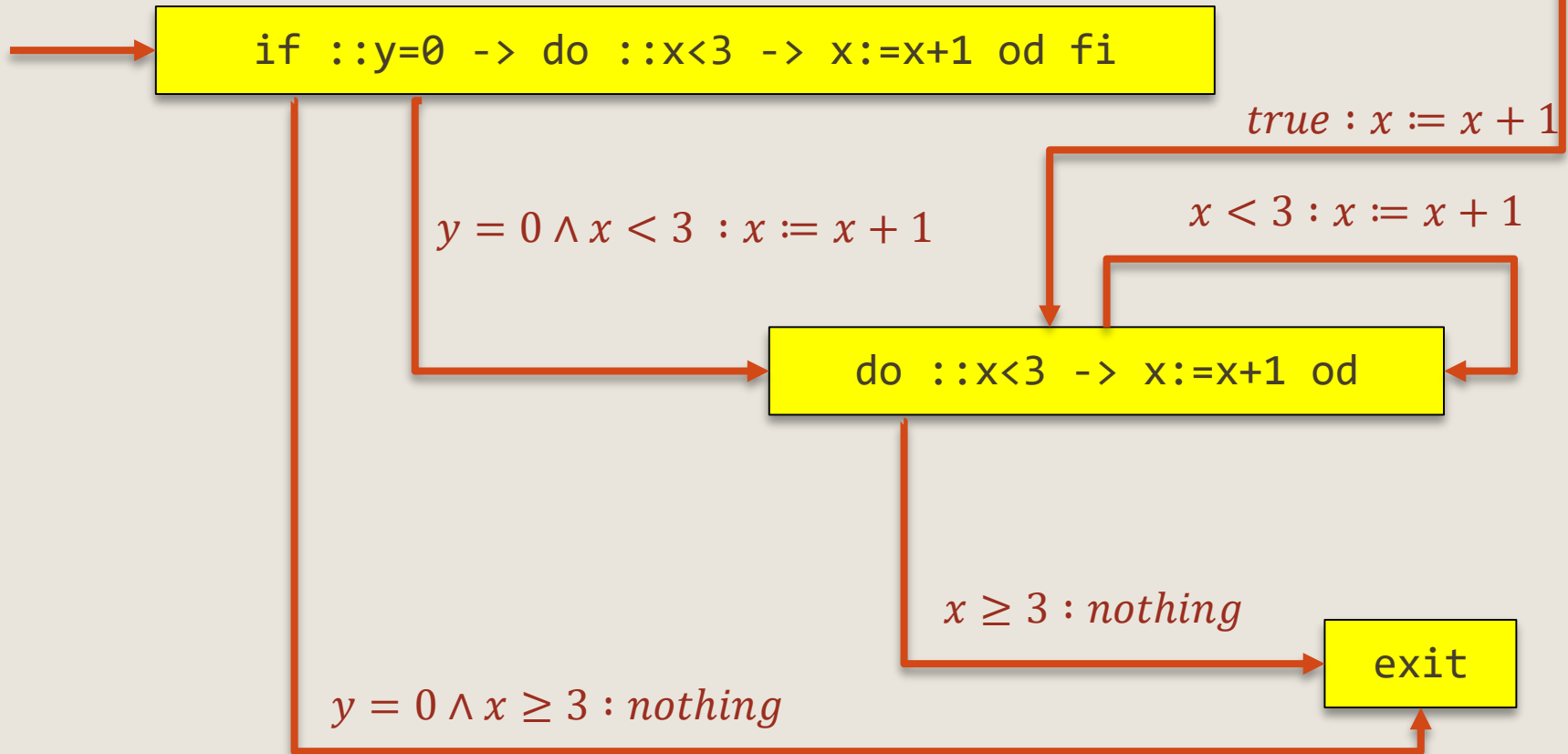
```
if  
  ::  $x > 1 \rightarrow y := x + y$   
  :: true  $\rightarrow x := 0; y := x$   
fi
```





דוגמה: לפעמים אפשר להגיע גם לפקודה הראשונה בתוך תנאי

$x := x + 1 ; \text{do} :: x < 3 \rightarrow x := x + 1 \text{ od}$





Promela

- השפה מכילה מבנים נוספים שלא פרטנו:
- קטעים אטומיים משוכללים – לא רק סדרות של השמות
- מערכים ומבני נתונים נוספים
- יצירה דינאמית של תהליכים
- ועוד...
- הסמנטיקה מוגדרת ע"י תרגום למערכות ערוצים ודרכן למערכות מעברים
- תיאור מלא של התחביר והסמנטיקה המלאים של Promela אפשר למצוא בספרות על SPIN (בשקף הבא)

מקורות נוספים ללימוד SPIN

<http://spinroot.com/spin/whatispin.html>

במעבדות המחלקה מותקנת התוכנה spin שהיא מימוש אלגוריתמי בדיקת המודל עצמם, והתוכנה ispin העוטפת את spin בממשק גרפי המאפשר קביעת פרמטרים באופן נוח ומציג את תוצאות הריצה בדרך שקל יותר להבין.

